

Private Lending for Aleo

Universal lending infrastructure for Aleo, built in Leo by the team behind OWN and inspired by PWN's bilateral, oracle-free lending mechanics.

APPLICANT	TIMELINE	GRANT REQUEST	DELIVERABLE
PWN DAO Foundation Team behind OWN · PWN protocol lineage	~4 months four milestones	\$85,000 mainnet launch	Protocol on mainnet programs · SDK · reference app

The applicant is the **PWN DAO Foundation**. The team previously developed PWN, a peer-to-peer lending protocol deployed on 13 EVM chains. The team now operates as [OWN](#), building crypto-backed mortgages on top of PWN's lending infrastructure. Lenders and borrowers agree on fixed loan terms directly, without price oracles or price-based liquidations. This bilateral model fits a privacy-first ZK chain particularly well, because individual loans stay isolated instead of depending on global liquidity, utilisation and liquidation state.

This grant funds **universal lending infrastructure for Aleo**, designed around PWN v1.5's lending mechanics: funded Elastic Proposals as the first offer type, private bilateral lending, a modular Hub and Config layer, a TypeScript SDK and a reference application, **with a mainnet launch as the final milestone**. A feasibility PoC of the core programs is already deployed and exercised on the Aleo testnet (see *Current status*).

Team

Josef Jelacic — Founder of PWN DAO and OWN. [LinkedIn](#)

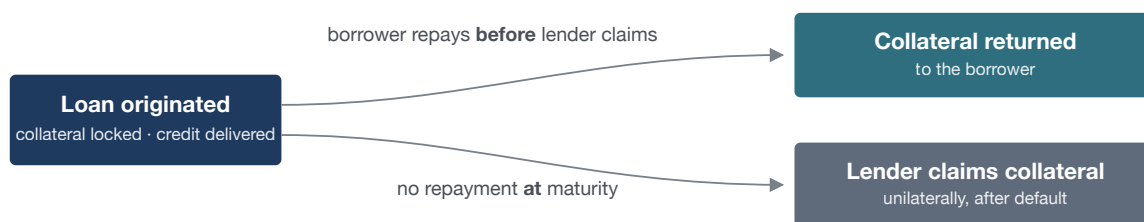
Šimon Kozák — Founding Engineer & CTO at PWN DAO and OWN. [LinkedIn](#) · [GitHub](#)

Jan Mikšík — Engineer, 4 years of building PWN's DeFi lending application. [LinkedIn](#) · [GitHub](#)

Protocol model

Each loan fixes the credit asset and amount, the collateral asset and amount, the repayment amount and the maturity. If the borrower repays, collateral is returned. If the loan is unpaid at maturity, the lender can claim the collateral; until the lender does so, the borrower can still repay. There is no price feed, no health factor and no price-based liquidation, which makes the model a natural fit for minimising public disclosure on Aleo.

The protocol is written in Leo around Aleo's record and execution model, drawing on PWN's lending mechanics and modular architecture.



Two outcomes, decided only by the borrower's action and the clock. No third party, no price feed, no liquidation.

Current status

The feasibility work is done. The four core programs are deployed on the public Aleo testnet and the complete lifecycle (funded commitment, borrower draw, repayment, default claim, refund and repayment withdrawal) has been executed there with real proofs, collateral on the live `token_registry.aleo`, and privately paid fees.

Custody & funded commitments

Demonstrated on testnet.

Lender commits once; borrower draws without any lender transaction; every exit settles into a private record.

Privacy

Measured on real transactions.

No lender or borrower address in any lending transaction paid with a private fee, which the SDK will enforce; amounts, terms and IDs public by design.

Costs & timings

Within limits. 0.005–0.019 ALEO per call; proving 25–41 s on a laptop; confirmation in 1–4 blocks.

Not yet done: direct offers, USDCx execution against the live stablecoin, a complete test suite and external review. These are the subject of the milestones below.

Feasibility questions, answered by the PoC

- 01 **Can lender credit be committed once and drawn asynchronously without lender participation?** Yes. Draws are signed by the borrower only.

- 02 **Can custody safely support private → controlled state → private asset flows?** Yes, for native credits and token-registry assets; a premature default claim was rejected by network finalization. Escrow is protected against users, not against a collateral token's issuer powers; see Open questions.

- 03 **What is public, private, inferable or linkable in each transition?** Measured; see the privacy page.

- 04 **Does the model work within USDCx compliance and token-registry constraints?** Token registry: yes, live. USDCx: interfaces compile against the real bytecode; execution with freeze-list proofs is part of Milestone 1.

- 05 **Can Proposal, Loan and token programs compose within proving and transaction limits?** Yes. A draw is four transitions across three programs in one transaction, at about 6% of the constraint limit.

- 06 **Measured proving and execution times?** 25–41 s per call on a laptop; 0.005–0.019 ALEO per call; confirmation in 1–4 blocks.

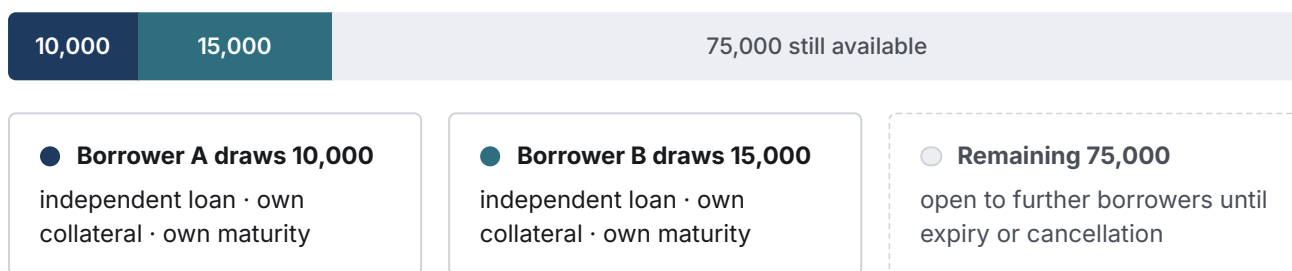
Elastic Proposal: the default offer type

In PWN's EVM implementation, an [elastic proposal](#) lets one lender publish a single reusable offer with a credit limit, a minimum draw and a fixed credit-to-collateral ratio. Borrowers draw from it independently; each draw is its own bilateral loan with its own collateral and maturity. It is not a pool: no shared position, no mutualised risk.

Proposal versus loan. A proposal is an offer and its available funding; each accepted draw creates a separate loan with its own collateral, repayment obligation and maturity. Proposal expiry stops new draws, not existing loans. Cancelling a proposal returns unused funding without cancelling active loans.

Lender commits **100,000 USDCx**

min draw **5,000**

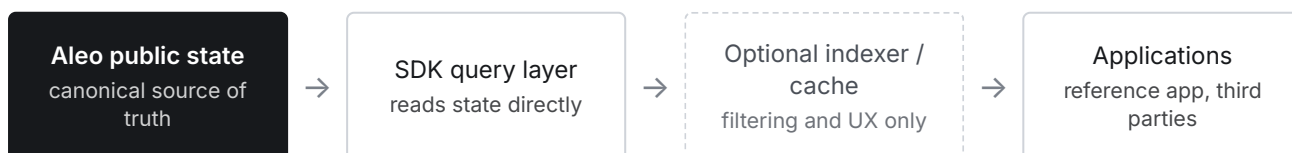


Why this is the default on Aleo

An Aleo transaction has a single signer and a program cannot spend a private record on a counterparty's behalf, so our asynchronous acceptance flow commits capital in advance. Elastic proposals make every offer backed by real capital and allow discovery from public state without a proprietary backend. Multiple draws reuse one funded offer. The lender's address stays private in the lending calls, while each loan publicly references the commitment it draws from. Direct offers use the same mechanism, with acceptance restricted to a designated borrower through a private proof of authorisation.

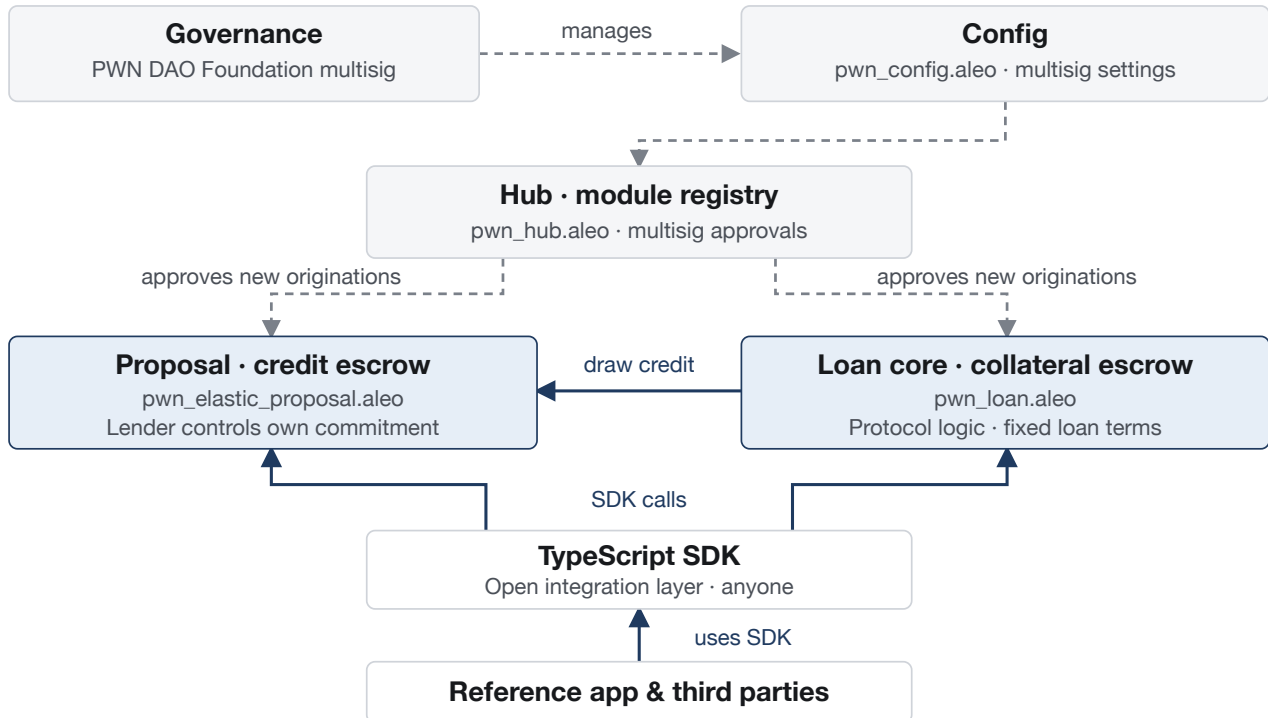
Discovery and indexing

Public Elastic Proposals remain canonical in Aleo public state. The SDK queries that state directly; an optional indexer or cache improves filtering and UX but is never a trusted source of truth. The wallet and SDK must discover private token records and securely retain the authorisation data needed to recover and manage loan positions.



Architecture

Target production architecture. The grant completes and audits these module boundaries before the core is deployed without upgradeability.



Dashed arrows: governance and module approval. Solid arrows: calls between layers. Asset transfers follow the loan lifecycle.

LAYER	PROGRAM	CONTROLLED BY	CAN AFFECT EXISTING LOANS
Governance	—	PWN DAO Foundation multisig	No
Registry & config	pwn_hub.aleo pwn_config.aleo	multisig	No: approves modules for <i>new</i> originations only
Proposal	pwn_elastic_proposal.aleo	lender	Own commitment only
Loan core	pwn_loan.aleo	protocol logic	Terms fixed at origination
Integration	TypeScript SDK	anyone	—
Application	Reference app, third parties	anyone	—

Governance can decide what may be created next. It can never alter, seize or redirect a loan that already exists.

Hub. Registers approved modules for new loan originations: `pwn_loan.aleo` → `ACTIVE_LOAN`, `pwn_elastic_proposal.aleo` → `PROPOSAL_MODULE`. Future installments, crowdfunding and refinancing arrive as new versioned modules.

Config. Minimal protocol-wide settings: protocol fee (zero in V1), fee recipient, multisig authority, and an emergency stop for new originations. Loan-specific economic terms are fixed at creation and cannot be changed later through Config.

After audit. The production core will be deployed without upgradeability after the external audit and resolution of material findings. New functionality is introduced through new modules; existing loans remain in the immutable programs holding their funds. Third-party token programs may retain their own issuer and upgrade powers.

Universal and extensible lending

Elastic Proposal is the first offer type, not the limit of the framework. Following PWN v1.5, the target architecture separates proposal validation and product rules from shared custody and settlement. Additional proposal types that fit the agreed interfaces can be introduced as new modules without changing the deployed core.

Installment products, crowdfunding and refinancing can build on these extension points.

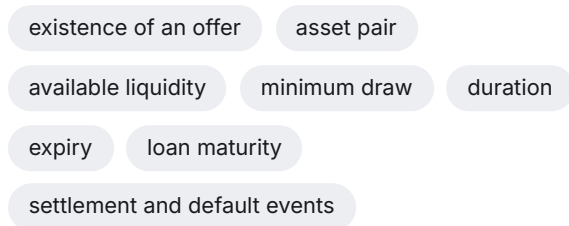
PWN v1.5 also provides borrower and lender hooks around loan creation and repayment, enabling integrations such as refinancing and external lender vaults. Equivalent Aleo integration points are a future extension, subject to privacy, authorisation and proving limits; hooks are not part of the demonstrated PoC.

Privacy model, measured on testnet

V1 behaviour as measured on the public Aleo testnet with the PoC programs, from confirmed transactions re-fetched from the network. What remains linkable is listed, not hidden.

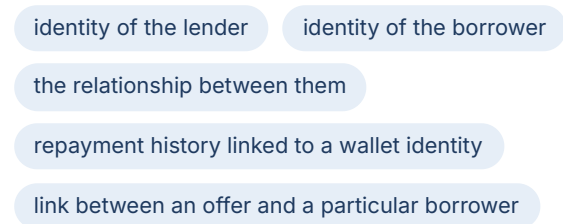
PUBLIC BY DESIGN

Required for the protocol to function, not a compromise.



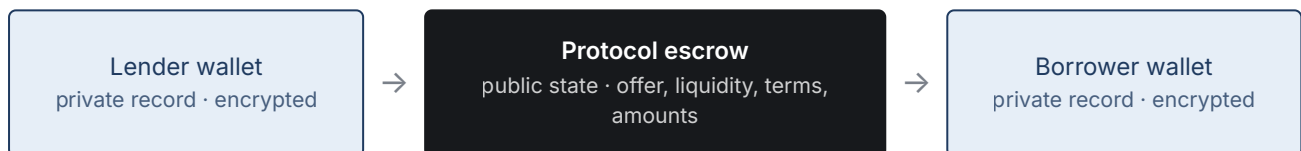
PRIVATE IN V1

Identities are not written to protocol public state; loan activity remains visible under public identifiers.



Direct offers can keep more terms private.

WHERE THE BOUNDARY SITS



Value passes through program-controlled public state; ownership on both ends stays in encrypted records. Amounts are visible in the middle; that is the leakage measured below.

Known leakage, measured

LEAK	MEASURED ON TESTNET	MITIGATION
Collateral amount	Public in the loan witness and in the token escrow delta	Normalised denominations; commitment-based private accounting in a later version
Draw amount	Public in the loan witness; delta in available liquidity per draw	Normalised draw steps
Lender / borrower address	Not present in any of the six lending flows when fees are paid from a private record; only program addresses appear. A public fee would name the payer.	Private fees, validated; the earlier public shielding transaction remains visible
Commitment → loan link	Public by construction (commitment ID inside the loan witness)	The link is public; it does not by itself reveal the lender's address
Temporal correlation	88 blocks between the borrower's shielding and the draw in the test run	Documented rather than concealed

3

Reference application

\$15k

3 weeks

SCOPE Lender and borrower application on top of the SDK: wallet connection, offer publishing and browsing, draw, repayment, default claim, position overview from private records, selective disclosure of a loan to a chosen party.

VALIDATION External testers complete the full lender and borrower journeys on testnet without help from the team; usability findings addressed; the application is published with usage documentation.

4

Security review & mainnet launch

\$20k

4 weeks

SCOPE External audit of the core programs, resolution of material findings, non-upgradeable mainnet deployment, SDK published as a package.

VALIDATION All critical invariants covered by tests; external audit completed and material findings resolved before mainnet deposits; at least one real loan originated, repaid and settled on mainnet end to end by an external user; deployment addresses and immutable constructor rules published.

If Milestone 1 finds a blocker that cannot be resolved, for example in USDCx compliance behaviour, we publish the findings and the privacy analysis as an open report for the ecosystem, and the remaining milestones are not claimed.

Open questions

Licensing

Does the Developer Grants Program require a specific open-source licence such as Apache-2.0, or is GPL-3.0 acceptable?

Collateral policy

Token issuers with public-burn, freeze or pause powers can affect a loan escrow. The current multi-token implementation checks a Hub asset allowlist when opening offers and accepting draws. Which issuer powers should V1 accept, and how should an escrow deficit affect positions in that token?

Out of scope

V1 is deliberately narrow: bilateral loans with terms fixed at origination. No oracles, no price-based liquidation, no pooled lending, no bridging to PWN's EVM deployments, no KYC or credit scoring. Installments, crowdfunded funding and refinancing are planned extensions. Protocol fees are zero. V1 targets a mainnet launch; the extensions follow later.

After the grant

The protocol becomes part of the PWN DAO Foundation's maintained infrastructure, with the team behind OWN maintaining the SDK, reference application and integrations. The deployed core remains immutable. The SDK is published as an open-source package for third-party integrators.

Possible extensions

Installment loans

Crowdfunded funding: several lenders fund one loan, each holding a private position

Refinancing

Private collateral amounts

Borrower and lender integration hooks

Existing references

OWN

ownlabs.co

Current team and product reference: longer-term crypto-backed lending and onchain mortgage infrastructure, building on PWN's lending primitives.

PWN Protocol v1.5

[v1.5-withdraw-restriction](#)

Architecture reference: Hub, Config, modular products and proposals, installments, crowdfunding and borrower/lender hooks.

PWN Elastic Proposal

dev-docs.pwn.xyz/smart-contracts/core/smart-contract-reference/proposals/simple-loan-proposal/elastic-proposal

Reference semantics of the elastic proposal: credit limit, minimum draw, credit-to-collateral ratio, expiry and per-draw loan creation, as implemented on EVM.

ProvableHQ compliant stablecoin / USDCx

github.com/ProvableHQ/compliant-stablecoin

Public and private token flows, freeze-list compliance, role administration, multisig upgrades, USDCx integration.

These are design references; the Aleo programs are written from scratch in Leo. Licensing is checked separately.